# SAM Rover Fleet — Technical Deployment & Provisioning SOP

**Document Version:** 2.0.0 (Alpha Build)
**Target Architecture:** Raspberry Pi 4 Model B / Raspberry Pi 5 (ARM64 / aarch64)
**Operating Systems:** Ubuntu 24.04 LTS / Ubuntu 26.04 LTS (Resolute Raccoon) / Debian Bookworm 64-bit
**Corpus / Repository:** `https://github.com/Abhijeet226/sam.git` (Branch: `alpha`)

---

## 1. Executive Summary & Architecture Overview

SAM (Smart Autonomous Mobile Rover) runs a high-throughput, low-latency teleoperation and autonomy daemon powered by:
- **FastAPI & Uvicorn**: Async REST API and WebSocket telemetry streaming at up to 20Hz.
- **Kernel Character Device `lgpio`**: Shared `/dev/gpiochip0` handle for zero-overhead hardware PWM motor control, microsecond ultrasonic polling, and directional LED actuation.
- **I2C Sensor Bus**: MPU-6050 / ICM-20948 IMU reading with real-time Complementary Filter sensor fusion.
- **Hardware Lease Arbitrator**: Enforces multi-operator safety. Unclaimed rovers unconditionally isolate all motor and actuator outputs.

---

## 2. Hardware Pre-Flight Checklist

Before provisioning software on any physical rover, ensure the following hardware connections:

| Subsystem | Component | BCM Pin Mapping | Interface |
|---|---|---|---|
| **Motors (Left)** | L298N H-Bridge | ENA: GPIO 17, IN1: GPIO 22, IN2: GPIO 27 | Hardware PWM / GPIO Out |
| **Motors (Right)** | L298N H-Bridge | ENB: GPIO 18, IN3: GPIO 23, IN4: GPIO 24 | Hardware PWM / GPIO Out |
| **Ultrasonic Front** | HC-SR04 | Trigger: GPIO 10, Echo: GPIO 9 (Voltage divider!) | GPIO Out / In |
| **Ultrasonic Rear** | HC-SR04 | Trigger: GPIO 11, Echo: GPIO 5 | GPIO Out / In |
| **Ultrasonic Left** | HC-SR04 | Trigger: GPIO 6, Echo: GPIO 13 | GPIO Out / In |
| **Ultrasonic Right** | HC-SR04 | Trigger: GPIO 19, Echo: GPIO 26 | GPIO Out / In |
| **Directional LEDs** | Indicators | Front: 12, Rear: 16, Left: 20, Right: 21 | GPIO Out (Active HIGH) |
| **Audio Buzzer** | Active Piezo | GPIO 25 | GPIO Out (Active HIGH) |
| **IMU** | MPU-6050 / 9250 | SDA: GPIO 2, SCL: GPIO 3 | I2C Bus 1 (Addr 0x68 / 0x69) |
| **Vision** | USB Webcam / CSI | `/dev/video0` (V4L2 compliant) | USB 2.0 / 3.0 / MIPI CSI |

> [!WARNING]
> HC-SR04 Echo pins output 5V logic. Ensure a resistor voltage divider (e.g., 1kΩ / 2kΩ) is placed between the sensor Echo pin and the Raspberry Pi GPIO input (3.3V tolerance) to prevent board damage.

---

## 3. Automated Single-Node Provisioning (Step-by-Step)

Log into the rover via SSH:

```
ssh <username>@<rover-ip>
# Default password: raspberry
```

### Step 3.1: Resolve System Dependencies & Fix Broken Locks

Ubuntu aarch64 installations frequently have pre-installed package locks or testsuite conflicts. Run the following clean-up sequence:

```
# 1. Clear any stuck dpkg configurations
sudo dpkg --configure -a

# 2. If encountering package version locks (e.g. libpython3.*-testsuite):
sudo dpkg --remove --force-all libpython3.14-testsuite curl 2>/dev/null || true

# 3. Update repository indices and fix dependencies
sudo DEBIAN_FRONTEND=noninteractive apt update
sudo DEBIAN_FRONTEND=noninteractive apt --fix-broken install -y

# 4. Install essential Linux kernel hardware packages & build utilities
sudo DEBIAN_FRONTEND=noninteractive apt install -y \
  python3-lgpio \
  liblgpio1 \
  python3-smbus2 \
  python3-opencv \
  python3-yaml \
  python3-pip \
  python3-venv \
  git \
  curl \
  network-manager
```

### Step 3.2: Verify Kernel Devices

Confirm kernel nodes exist:

```
ls -la /dev/gpiochip* /dev/i2c-1 /dev/video0
```

*Expected Output:*
- `/dev/gpiochip0` (Pi 4 BCM GPIO controller)
- `/dev/i2c-1` (Hardware I2C bus)
- `/dev/video0` (V4L2 camera device)

### Step 3.3: Clone Repository & Create Virtual Environment

```
# 1. Create target directory
cd ~
git clone -b alpha https://github.com/Abhijeet226/sam.git sam
cd ~/sam

# 2. Create Python virtual environment with --system-site-packages
# CRITICAL: --system-site-packages inherits pre-compiled python3-lgpio & python3-opencv!
python3 -m venv --system-site-packages .venv

# 3. Install core ASGI web dependencies inside virtual environment
./.venv/bin/pip install --upgrade pip
./.venv/bin/pip install fastapi "uvicorn[standard]" websockets pydantic jinja2 psutil
```

### Step 3.4: Test Hardware Subsystems in Standalone Mode

Run an inline hardware smoke test:

```
./.venv/bin/python -c "
import lgpio, smbus2, cv2, yaml, fastapi, pydantic
print('Core Python modules successfully validated!')

from core.hal.gpio_chip import gpio_chip_manager
chip = gpio_chip_manager.get_chip()
print(f'Kernel gpiochip handle opened: {chip}')
gpio_chip_manager.close_chip()
"
```

---

## 4. Systemd Daemon Installation (Auto-Start on Boot)

Create `/etc/systemd/system/sam.service` replacing `<username>` with the local user (e.g. `sam2`, `sam3`, or `pi`):

```
[Unit]
Description=SAM Rover Teleoperation Web & Autonomy Server
After=network.target network-online.target
Wants=network-online.target

[Service]
Type=simple
User=root
WorkingDirectory=/home/<username>/sam
ExecStart=/home/<username>/sam/.venv/bin/python -m uvicorn server.app:app --host 0.0.0.0 --port 8
Restart=always
RestartSec=3
TimeoutStopSec=2s
Environment=PYTHONUNBUFFERED=1

[Install]
WantedBy=multi-user.target
```

Apply and start the daemon:

```
sudo systemctl daemon-reload
sudo systemctl enable sam.service
sudo systemctl restart sam.service
sudo systemctl status sam.service --no-pager
```

---

## 5. Multi-Rover Fleet Deployment Automation

For provisioning 5 to 50+ rovers across a network simultaneously, utilize the Python deployment automation script from the deployment workstation:

## `deploy_fleet.py` Script

```
import paramiko
from scp import SCPClient
import os, sys, time

ROVERS = [
    {"host": "192.168.0.144", "user": "sam2", "pass": "raspberry"},
    {"host": "192.168.0.187", "user": "sam3", "pass": "raspberry"},
    # Add additional rovers here
]

FILES = [
    ("core/hal/gpio_chip.py", "core/hal/gpio_chip.py"),
    ("core/hal/motors.py", "core/hal/motors.py"),
    ("core/hal/ultrasonic.py", "core/hal/ultrasonic.py"),
    ("core/hal/leds.py", "core/hal/leds.py"),
    ("core/hal/buzzer.py", "core/hal/buzzer.py"),
    ("core/hal/camera.py", "core/hal/camera.py"),
    ("core/hal/imu.py", "core/hal/imu.py"),
    ("core/hardware_manager.py", "core/hardware_manager.py"),
    ("server/routes/hardware.py", "server/routes/hardware.py"),
    ("server/routes/teleop.py", "server/routes/teleop.py"),
    ("server/routes/video.py", "server/routes/video.py"),
    ("server/static/js/app.js", "server/static/js/app.js"),
    ("server/static/css/style.css", "server/static/css/style.css"),
    ("config.yaml", "config.yaml"),
]

for rover in ROVERS:
    print(f"\n========================================")
    print(f"Deploying to {rover['user']}@{rover['host']}...")
    print(f"========================================")
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    try:
        client.connect(rover['host'], port=22, username=rover['user'], password=rover['pass'], ti
        with SCPClient(client.get_transport()) as scp:
            for src, dst in FILES:
                rem_path = f"/home/{rover['user']}/sam/{dst}"
                scp.put(src, remote_path=rem_path)

        # Set permissions & restart systemd
        sudo_cmd = f"echo {rover['pass']} | sudo -S systemctl restart sam.service"
        client.exec_command(sudo_cmd)
        time.sleep(2)

        # Verify status
        _, stdout, _ = client.exec_command("systemctl is-active sam.service")
        status = stdout.read().decode().strip()
        print(f"[{rover['host']}] sam.service status: {status}")
        client.close()
```

```
    except Exception as e:
        print(f"Failed to deploy to {rover['host']}: {e}")
```

## 6. Post-Deployment Verification & Health Diagnostics

Execute an HTTP health check on each rover:

```
# 1. Health readiness endpoint
curl -s http://<rover-ip>:8000/readyz
# Expected: {"status":"ready","ready":true,"checks":{"supervisor_thread":true,"sensor_thread":tru

# 2. Aggregated telemetry inspection
curl -s http://<rover-ip>:8000/api/hardware/telemetry
```

## 7. Rollback & Maintenance Plan

If a software update requires rollback:

```
cd ~/sam
git checkout main   # or specific stable commit tag
sudo systemctl restart sam.service
```

To view real-time daemon logs:

```
journalctl -u sam.service -f -n 50
```